

Sorting

March 15, 2024

Counting Sort

- ▶ Sorting algorithm introduced by Harold H. Seward in 1954.
- ▶ Operates in linear time.
- ▶ Efficient for sorting integers with a known range.

Counting Sort

1. Find the range of input elements.
2. Create an auxiliary array (count array) to store element counts.
3. Traverse the input array and update the count array.
4. Compute the prefix sums over the count array.
5. for $i = n - 1$ downto 0
 - ▶ Let j be the i -th input element
 - ▶ There are $\text{count}[j]$ input elements that are $\leq$ this j ; so, this j goes to output location $\text{count}[j] - 1$.
 - ▶ decrement $\text{count}[j]$.

Counting Sort

- ▶ Time complexity: $O(n + k)$
- ▶ n - number of elements in the array
- ▶ k - range of input values
- ▶ Stable sort - maintains relative order of equal elements between input and output

Counting Sort

- ▶ Advantages:
 - ▶ Linear time complexity for suitable cases.
 - ▶ Stable sorting.
- ▶ Limitations:
 - ▶ Practical for small ranges, less practical for large ranges.
 - ▶ Limited to integer data.

Example

- ▶ Consider the input array: $[4_a, 2_a, 3_a, 1_a, 4_b, 2_b, 3_b, 4_c, 1_b, 2_c]$.
- ▶ The subscripts are to distinguish elements with the same key values
- ▶ Range: 1 to 4. (subscripts are not part of keys)
- ▶ Count array initialization: $[0, 0, 0, 0]$.

Step 1: Count Occurrences

- ▶ Traverse the input and update the count array.
- ▶ Input array: $[4_a, 2_a, 3_a, 1_a, 4_b, 2_b, 3_b, 4_c, 1_b, 2_c]$.
- ▶ Count array: $[2, 3, 2, 3]$.

Step 2: Cumulative Counts

- ▶ Compute prefix sums over the count array.
- ▶ Prefix sums of the count array: $[2, 5, 7, 10]$.

Step 3: Build Sorted Output

- ▶ Use the prefix sums array to build the sorted output array.
- ▶ 2_c is the last element; there are 5 that are $\leq 2_c$; so, 2_c goes to output location 4.
- ▶ Prefix sums array is $[2, 4, 7, 10]$ now.
- ▶ 1_b is the 2nd last element; there are 2 that are $\leq 1_b$; so, 1_b goes to output location 1.
- ▶ Prefix sums array is $[1, 4, 7, 10]$ now.
- ▶ 4_c is the 3rd last element; there are 10 that are $\leq 4_c$; so, 4_c goes to output location 9.
- ▶ Prefix sums array is $[1, 4, 7, 9]$ now.
- ▶ ...
- ▶ Sorted output array: $[1_a, 1_b, 2_a, 2_b, 2_c, 3_a, 3_b, 4_a, 4_b, 4_c]$.

Counting Sort - C Code

```
#include <stdio.h>

void countingSort(int arr[], int n) {
    // Find the maximum and minimum values in the array
    int max = arr[0], min = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
        if (arr[i] < min) {
            min = arr[i];
        }
    }

    // Create the count array and initialize to 0
    int range = max - min + 1;
    int count[range];
    for (int i = 0; i < range; i++) {
        count[i] = 0;
    }
}
```

Counting Sort - C Code

```
// Count the occurrences of each element
for (int i = 0; i < n; i++) {
    count[arr[i] - min]++;
}
// Update the count array with prefix sums
for (int i = 1; i < range; i++) {
    count[i] += count[i - 1];
}
// Create the sorted output array
int output[n];
for (int i = n - 1; i >= 0; i--) {
    output[count[arr[i] - min] - 1] = arr[i];
    count[arr[i] - min]--;
}
// Copy the sorted output back to the original array
for (int i = 0; i < n; i++) {
    arr[i] = output[i];
}
}
```

Counting Sort - C Code

```
int main() {  
    int arr[] = {4, 2, 1, 7, 5, 3, 8, 6};  
    int n = sizeof(arr) / sizeof(arr[0]);  
  
    printf("Original array: ");  
    for (int i = 0; i < n; i++) {  
        printf("%d ", arr[i]);  
    }  
  
    countingSort(arr, n);  
  
    printf("\nSorted array: ");  
    for (int i = 0; i < n; i++) {  
        printf("%d ", arr[i]);  
    }  
  
    return 0;  
}
```

Radix Sort

- ▶ Radix sort is a non-comparative sorting algorithm.
- ▶ It operates by distributing elements into buckets based on individual digits or radix.

Algorithm Overview

1. **Initialization:** Determine the maximum number of digits in the given sequence of numbers.
2. **Passes:** Perform passes through the digits from LSD to MSD or vice versa.
3. **Counting Sort:** Apply Counting Sort to the current sequence based on the current digit being processed.
4. **Repeat:** Output of one pass is input to the next.

Variants of Radix Sort

- ▶ **LSD Radix Sort:** Starts sorting from the rightmost digit.
(Preferred.)
- ▶ **MSD Radix Sort:** Starts sorting from the leftmost digit.
- ▶ MSD Radix Sort gives 10 sequences corresponding to MSD 0, 1, ..., 9.
- ▶ Each has to be sorted recursively and combined.
- ▶ LSD Radix sort has only one sequence to handle all the time.

LSD Radix Sort

- ▶ E.g., Stack of 1000 answer papers to sort with rollnumbers from 000 to 999
- ▶ Form 10 buckets corresponding to digits 0 to 9
- ▶ Place papers into buckets face down based on LSD; 247 goes to bucket-7
- ▶ Flip the buckets face up; stack bucket-0 on bucket-1 on bucket-2 etc.
- ▶ Use 2nd LSD for the next pass, which also end with one stack.
- ▶ Repeat.

Performance

- ▶ Radix sort is effective for sorting integers or fixed-length strings.
- ▶ Linear time complexity makes it efficient for large datasets.
- ▶ Performance can degrade with numbers having varying numbers of digits.

Example

- ▶ **Initial array:**
- ▶ 170, 048, 075, 090, 802, 024, 002, 066, 273, 300, 497, 189, 921
- ▶ **Pass 1:** Sort by the units place.
- ▶ 170, 090, 300, 921, 802, 002, 273, 024, 075, 066, 497, 048, 189
- ▶ **Pass 2:** Sort by the tens place.
- ▶ 300, 802, 002, 921, 024, 048, 066, 170, 273, 075, 189, 090, 497
- ▶ **Pass 3:** Sort by the hundreds place.
- ▶ 002, 024, 048, 066, 075, 090, 170, 189, 273, 300, 497, 802, 921

Performance and Complexity

- ▶ Linear time complexity for integers with fixed-length.
- ▶ Degrades with varying numbers of digits.

Radix Sort - C Code

```
#include <stdio.h>

// Function to find the maximum number
// to determine the number of digits
int getMax(int array[], int n) {
    int max = array[0];
    for (int i = 1; i < n; i++)
        if (array[i] > max)
            max = array[i];
    return max;
}
```

Radix Sort - C Code

```
// Using counting sort to sort the elements
// based on significant places
void countingSort(int array[], int n, int exp) {
    const int RANGE = 10; // Assuming base 10 (decimal system)
    int output[n];
    int count[RANGE] = {0};
        // Count occurrences in count[]
    for (int i = 0; i < n; i++)
        count[(array[i] / exp) % RANGE]++;
        // Cumulative count to get actual position
    for (int i = 1; i < RANGE; i++)
        count[i] += count[i - 1];
        // Build the output array
    for (int i = n - 1; i >= 0; i--) {
        output[count[(array[i] / exp) % RANGE] - 1] = array[i];
        count[(array[i] / exp) % RANGE]--;
    }

        // Copy the output array to the original array
    for (int i = 0; i < n; i++)
        array[i] = output[i];
}
```

Radix Sort - C Code

```
// Radix Sort function
void radixSort(int array[], int n) {
    // Find the maximum number to determine the number of digits
    int max = getMax(array, n);

    // Perform counting sort for every digit place
    for (int exp = 1; max / exp > 0; exp *= 10)
        countingSort(array, n, exp);
}

// Print an array
void printArray(int array[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", array[i]);
    printf("\n");
}
```

Radix Sort - C Code

```
int main() {  
    int array[] = {170, 45, 75, 90, 802, 24, 2, 66};  
    int n = sizeof(array) / sizeof(array[0]);  
  
    printf("Original array: ");  
    printArray(array, n);  
  
    // Perform radix sort  
    radixSort(array, n);  
  
    printf("Sorted array: ");  
    printArray(array, n);  
  
    return 0;  
}
```

Recursion

- ▶ C supports recursive functions.
- ▶ Example 1: printing a number as a character string.
- ▶ Example 2: quicksort algorithm.
- ▶ Recursion is especially convenient for recursively defined data structures.

Recursion Example: Printing a Number

```
#include <stdio.h>

void printd(int n) {
    if (n < 0) {
        putchar('-');
        n = -n;
    }
    if (n / 10)
        printd(n / 10);
    putchar(n % 10 + '0');
}
```

Recursive Quicksort

```
void qsort(int v[], int left, int right) {  
    int i, last;  
    void swap(int v[], int i, int j);  
  
    if (left >= right)  
        return;  
  
    swap(v, left, (left + right) / 2);  
    last = left;  
  
    for (i = left + 1; i <= right; i++)  
        if (v[i] < v[left])  
            swap(v, ++last, i);  
  
    swap(v, left, last);  
  
    qsort(v, left, last - 1);  
    qsort(v, last + 1, right);  
}
```